ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/328891908

Genetic Algorithm Approach for UAV Persistent Visitation Problem

Conference Paper · September 2018

DOI: 10.1115/DSCC2018-8950

CITATIONS
5

READS
196

4 authors, including:



Alexander Von Moll
United States Air Force Research Laboratory
101 PUBLICATIONS **1,163** CITATIONS
SEE PROFILE



Krishna Kalyanam
National Aeronautics and Space Administration
90 PUBLICATIONS **840** CITATIONS
SEE PROFILE



Satyanarayana Manyam
Infoscitex Corporation
101 PUBLICATIONS **1,099** CITATIONS
SEE PROFILE

All content following this page was uploaded by Alexander Von Moll on 13 November 2018.

The user has requested enhancement of the downloaded file.

GENETIC ALGORITHM APPROACH FOR UAV PERSISTENT VISITATION PROBLEM

Alexander L. Von Moll*
David W. Casbeer
Controls Science Center of Excellence
Air Force Research Laboratory
WPAFB, OH 45433

Krishna Kalyanam
Satyanarayana G. Manyam
InfoSciTex Corporation, a DCS Company
4027 Colonel Glenn Hwy, Ste 210
Dayton, OH 45431

ABSTRACT

We employ a genetic algorithm approach to solving the persistent visitation problem for UAVs. The objective is to minimize the maximum weighted revisit time over all the sites in a cyclicly repeating walk. In general, the optimal length of the walk is not known, so this method (like the exact methods) assume some fixed length. Exact methods for solving the problem have recently been put forth, however, in the absence of additional heuristics, the exact method scales poorly for problems with more than 10 sites or so. By using a genetic algorithm, performance and computation time can be traded off depending on the application. The main contributions are a novel chromosome encoding scheme and genetic operators for cyclic walks which may visit sites more than once. Examples show that the performance is comparable to exact methods with better scalability.

PROBLEM STATEMENT

The persistent visitation problem can be described as a set of physical sites that require periodic surveillance by a single UAV. Let there be m such sites, and let $w_i,\ i = 1,\ldots,m$ be the weight associated with each site corresponding to some relative importance. Sites with larger weights ought to be visited more often than sites with smaller weight. We are interested in constructing a *kernel* (or closed walk) of finite length, N, that is repeated *ad infinitum*. Every site must be visited at least once in the kernel, however repeat visits to sites are allowed, so $N \geq m$. The objective is, for a given N, find the kernel that minimizes the maximum weighted revisit time over all sites. Let the cost function be defined as $C(\kappa, w, D) = \max_i w_i \cdot R(i)$ where $D \in \mathbb{R}^{m\times m}$ is a distance matrix and $R(i)$ is the maximum time between successive visits to node i in the kernel κ. If Some site is visited only once in the kernel, the length of the closed walk is its $R(i)$.

Prior Work on Persistent Visitation

The persistent visitation problem fits into a broader class of Persistent Intelligence, Reconnaissance, and Surveillance (PISR) problems. These problems have been investigated heavily in part due to the proliferation of unmanned systems that can execute missions whose time far exceeds human capability. Solutions and solution methods for these problems are heavily dependent on the scenario and the notion of cost. For example, in [1] the scenario included a data delivery aspect wherein the "freshness" of the data delivered to a depot was incorporated into the cost. Moreover, the cost was based on an average reward as opposed to an upper bound on revisit frequency and dynamic programming is employed in the solution. Similarly, [2,3] considered freshness of data and revisit frequency, but the cost function had a min-max scheme and a branch-and-bound tree search was used. Weighted tours have also been considered for teams of autonomous agents in [4] wherein distributed algorithms for converging to optimal trajectories were discussed.

The most general description of the scenario of persistent visitation to weighted nodes can be found in [5]. Included are two approximate algorithms for solving the problem on a general graph. An exact algorithm was later developed in [6] which is useful for problems with roughly $m = 10$ nodes with $N \leq 2m$, after which the combinatorial nature of the problem causes the time required to compute a solution to be excessive.

Prior Work in Applying GAs to TSP

Although many good heuristic algorithms (cf. [7]) for solving the TSP have been constructed, there exists a sizable body of research in applying genetic algorithms to TSP. A good survey of the work done prior to the year 2000 is contained in [8]. One of the more prominent aspects of the research is in designing the crossover operator. In general, crossover operators can

*Corresponding author. alexander.von_moll@us.af.mil

be categorized into edge-based, position-based, and order-based. More recent work in applying GAs to TSP focuses on creating novel crossover and mutation operators [9, 10, 11]. Finally, the TSP was used as an example application for the introduction of the immune genetic algorithm (IGA) in [12] which includes an entirely new type operator.

TECHNICAL PRELIMINARIES

Although the present focus is on the real-world application of a UAV visiting geographic locations, we will be lax in our definition of what a *visit* means. Specifically, we allow identity transitions in the walk, which means that any site may be visited consecutively without affecting the cost (i.e. the time to travel zero distance is zero). We also assume that the vehicle has a fixed unit speed, thus time is equivalent to distance. The only other constraint on the walk is that each site must be visited at least once.

Upper Bound

It has been shown in [5] and [6] that the optimal tour with cost C^* for $N = m$ is simply the TSP tour. Since every site is visited exactly once, the revisit time for all of the nodes is the time to complete one iteration of the walk (i.e. $R(i) = \sum_{j=0}^{m-1} D_{j,j+1} + D_{m,0}\ \forall i$). Thus the cost is dominated by the node of maximum weight and the time to complete one iteration of the walk, the latter of which is minimized by the TSP. Because the optimal kernel length, N^*, is unknown, the approach in [6] is to start with $N = m$ and increment until $N = 2m$ (in general). The allowance of identity transitions means that the optimal cost is monotonic non-increasing w.r.t. N since in moving from N to $N+1$ any identity transition may be inserted without affecting the cost. Thus the TSP gives an upper bound on the optimal cost; we refer to this bound as $\bar{C}$. For the special case of equal weights, $w_i = w_j,\ i \neq j$, we have $N^* = m$ and the TSP yields the optimal cost over all N.

Lower Bound

Similarly, [5, 6] identify a useful lower bound which also holds for all N. It is given by $\underline{C} = \max_i \max_{j,j'} w_i(D_{i,j} + D_{j',i})$, which says that even if a particular site is visited after each other site the UAV still must travel to and from the furthest neighbor, respectively. In order for $C^* = \underline{C}$ to occur, the optimal walk must contain the edges that define $\underline{C}$ and be dominated by site that is common to both edges. Determining whether or not $C^* = \underline{C}$ for a given set of sites and weights may be as difficult as solving the problem in the first place. However, if the lower bound is encountered during the GA computation, then the GA should terminate immediately.

TABLE 1. CHROMOSOME SPECIFICATION

Section	Swaps	Added Nodes	Insert Indices
Length	m	$N-m$	$N-m$
Value	$\{0,\ldots,m\}$	$\{1,\ldots,m\}$	$\{0,\ldots,N-1\}$

GA FORMULATION

Throughout the remainder of the paper we use the terms cost and fitness interchangeably. Traditionally, GAs are designed to *maximize* fitness; however, since our problem involves minimization, the "best" fitness refers to the smallest value (i.e. the minimum cost).

Chromosome Description

One of the most important factors in the application of a GA to some problem is the encoding of candidate solutions into chromosomes. For the general TSP, several encoding schemes have been proposed including the canonic representation (where a chromosome is the tour itself), the ordinal representation [13], and the adjacency representation (which is mainly used with edge-based crossover operators) [8]. The encoding scheme is closely coupled with the genetic operators and has a big impact on whether or not the operators can be designed to preserve the feasibility of the chromosomes. Feasibility for the general TSP means that each site is visited exactly once, whereas in the persistent visitation problem feasibility means each site is visited at least once. For example, in the general TSP, an encoding for which operators can easily ensure feasibility is for the chromosome digits to represent swaps in a base tour that starts at $i = 1$ and visits each site in order. Having to handle infeasible chromosomes in a population is burdensome and should be avoided lest the GA run-time be poor. Another consideration for the encoding scheme is how to embed problem-specific knowledge into the structure of the chromosome. For the case of general TSP, an example could be if a particular edge is known *a priori* to be in the solution, then the chromosome ought to encode *where* this edge occurs (as opposed to *if* the edge occurs). Inclusion of this type of knowledge can drastically reduce the search space, making the GA more effective.

The chromosome can be thought of as a program whose input is the base walk that starts at $i = 1$ and visits each site in order, and whose output is the walk that it encodes. It consists of three sections which contain three different types of information. In general, c denotes a chromosome, a denotes a walk, and $[\cdot]$ denotes a zero-based accessor.

Definition 1 (Feasible Chromosome). *A chromosome is feasible if it encodes a walk wherein every site, $i = 1,\ldots,m$, is visited one or more times, and only those sites are visited.*

Remark. *Definition 1 does not preclude a feasible chromosome from encoding a walk containing identity transitions (i.e. consecutive visits to the same site).*

Table 1 contains the specification of the chromosome for the persistent visitation problem. We refer to the process by which the walk is computed from the chromosome as *decoding* (see Algorithm 1).

Algorithm 1 DECODE WALK FROM CHROMOSOME

```
1:  procedure DECODE(c, m, N)
2:      a ← [1,...,m,0,...]
3:      for i ← m−1,0 do                                  ▷ swaps
4:          if c[i] > 0 then
5:              Swap i and c[i]−1 in a
6:          end if
7:      end for
8:      for i ← m,N−1 do                                  ▷ inserts
9:          if a[c[i+N−m]] = 0 then
10:             a[c[i+N−m]] = c[i]
11:         else
12:             Insert c[i] at c[i+N−m] and shift right
13:             Delete left-most 0
14:         end if
15:     end for
16:     return a
17: end procedure
```

First, swaps are applied to the base walk starting at the mth digit of the chromosome and working backwards. The index or position of the digit in the chromosome indicates the index of the site to be swapped in the base walk. Meanwhile, the value of the digit in the chromosome indicates the other index which is to be swapped in the base walk. Swaps are applied in succession, so a particular site may be swapped with another multiple times. No swap is performed when the digit is zero. Following the swaps section is the added nodes section which is a string of sites to add to the reorganized base walk to get to a total walk length of N. Finally, the insert indices section is of the same length as the added nodes section; it specifies where in the walk to insert each site. The insertions are done in succession – thus repeats in the insert indices section are allowed. If an index is occupied, then everything from that point on in the walk is shifted right. In this case, the chromosome's length has increased by one, so the left-most occurrence of 0 is deleted to bring the chromosome back to the proper length.

Lemma 1. *If a chromosome, c, adheres to the specifications in Table 1, namely each section's length and set of values is respected, the chromosome must be feasible.*

Proof. According to Definition 1, it is sufficient to show that c encodes a walk, a, that visits every site at least once and no spurious sites are in the walk. In order to do so, we make use of the DECODE procedure given in Algorithm 1. First, we make the relationship between a and c explicit by stating $a = \text{DECODE}(c,m,N)$. Line 2 initializes the walk with exactly one visit to each site and right-pads the walk with zeros (which simply serve as placeholders). Entries in a are only reassigned if the current entry is zero (see Line 10). Otherwise, sites are inserted into the walk and only zero entries are deleted (see Line 13). Thus the original visits are preserved throughout the DECODE procedure. Finally, the only sites which are written over existing zeros or inserted into the walk (Lines 10 and 12, respectively) correspond to digits in c in the range of indices from m to $N-1$. According to Table 1 these digits must have a value in $\{1,\ldots,m\}$ – thus no spurious sites are added to the walk. □

The process in Algorithm 1 can be reversed; that is, every feasible walk of length less than or equal to N may be *encoded* as a chromosome. If a walk has length less than N, the mapping is not unique since the walk length can be made to equal N by inserting enough identity transitions. This is of particular importance for generating seed populations for the GA. Seeding the population with walks which are known *apriori* to have reasonable cost can drive the population to low cost solutions quickly. An obvious choice in this case is the solution to the TSP since it forms the upper bound of the cost function. Algorithm 2 specifies the process by which a walk is encoded as a chromosome.

Lemma 2. *If a walk a contains one or more visits to every site, $i = 1,\ldots,m$ and no visits to spurious sites, and $N \geq \text{LENGTH}(a)$ then the chromosome $c = \text{ENCODE}(a,N)$ is feasible.*

Proof. The proof is omitted due to its similarity to the proof of Lemma 1. □

Operators

This section describes each of the genetic operators and how they are used. The crossover and mutation operators are both very basic and could certainly be replaced by operators that are more tailored to this particular problem. In the case of selection, two generic operators are compared. Finally, an immunization operator is introduced for this problem for use within an IGA framework.

Crossover. The crossover scheme utilized here is based on sexual reproduction – two parents generate two children. Once the two parents are determined, the operation proceeds from $i = 1, \ldots, 2N-1$ selecting a corresponding digit from either the first parent, p_1, or second parent, p_2 according to a pre-selected, constant probability P_{cross}.

Algorithm 2 ENCODE WALK AS CHROMOSOME

procedure ENCODE(a, N)
  $\hat{a} \leftarrow$ REMOVEDUPLICATES(a)
  $m \leftarrow$ LENGTH($\hat{a}$)
  $c_i = 0,\ i = 1,\ 2N - m$
  $\hat{a}_{tmp} \leftarrow \hat{a}$
  for $i \leftarrow 0,\ m-1$ **do** ▷ reverse swaps
    $site \leftarrow i+1$
    $idx \leftarrow$ index of $site$ in $\hat{a}_{tmp}$
    if $idx\ != \ i$ **then**
      Swap i and idx in $\hat{a}_{tmp}$
      $c[i] \leftarrow idx+1$
    end if
  end for
  $ins \leftarrow 0$
  for $i \leftarrow 0,\ m-1$ **do** ▷ determine inserts
    if $i \geq m$ **or** $a[i]\ != \ \hat{a}[i]$ **then**
      Insert i into $\hat{a}$ and shift
      $c[m+ins] \leftarrow a[i]$
      $c[N+ins] \leftarrow i$
      $ins = ins+1$
    end if
  end for
  for $i \leftarrow$LENGTH(a), $N-1$ **do** ▷ pad to extend to N
    $site \leftarrow$ Random choice from $\{1,\ldots,m\}$
    $idx \leftarrow$ Random choice from indices occupied by $site$ in a
    $c[m+ins] = site$
    $c[N+ins] = i$
    $ins = ins+1$
    Insert $site$ into a at idx and shift right
  end for
  return c
end procedure

Lemma 3. *In crossover, if both parents are feasible chromosomes then both children must be feasible.*

Proof. Each digit in the children corresponds to a digit in the same position in one of the parents. Since both parents are feasible, the digits in both children must then also adhere to the specifications in Table 1 and are thus feasible by Lemma 1. □

Selecting a crossover probability $P_{cross} = 0.5$ means there is no bias in selecting a digit from either parent. In this case, both children will, statistically speaking, equally resemble both parents. However, when $0.5 < P_{cross} \leq 1$ the first child will more closely resemble p_1.

Mutation. The mutation operator is applied to each child after crossover occurs. It also proceeds digit by digit in the chromosome, changing the value of the digit according to a preselected, constant probability P_{mut}. The new value for the digit is selected randomly from the set of admissible values for the section in which the digit resides according to Table 1.

Lemma 4. *A feasible chromosome will remain feasible after mutation is applied.*

Proof. Since a digit may only mutate to a value specified in the corresponding section in Table 1, then by Lemma 1 the mutated chromosome must be feasible. □

Remark. *If identity transitions are not allowed in the walk (e.g. if* $1,2,2,3$ *were considered infeasible because of the repeated visit to* 2*), then this mutation operator may not result in feasible chromosomes.*

In fact, restricting the problem in this way could render much of a population infeasible. The removal of identity transitions could be handled in two ways: 1) chromosomes whose corresponding tours contain them could be penalized heavily in the cost function or 2) infeasible chromosomes could be removed completely. The penalization in the former case introduces the added complexity of tuning the penalization factor. In the latter case, there is the additional complexity of maintaining the population at its proper size and needing to replace infeasible chromosomes with feasible ones. Thus we allow identity transitions in this formulation for simplicity and ease of implementation.

Lemma 5. *If the initial GA population is comprised of feasible chromosomes, the individuals comprising the population must remain feasible for all generations.*

Proof. New chromosomes are only introduced to the population through crossover and are then mutated, thus by Lemmas 3 and 4, new chromosomes must be feasible if the initial population is feasible. □

Selection. Two different selection operators were considered: truncation and roulette wheel. Either operator can be used with this GA formulation and the results of using each selection scheme are given in the sequel. In truncation selection, the pool of individuals is cut in half (back to its original size) by selecting the fittest individuals. Prior to crossover the reproductive partners are assigned randomly; every chromosome participates in one crossover operation per generation. The parents are kept as a part of the population, so after reproduction there are twice as many individuals. Thus the fittest individual in the population is guaranteed to persist. Truncation selection has the nice property that the fittest individual in the last generation will have cost less than or equal to the fittest individual in the initial population. When the TSP solution is used in the seed population, the fittest individual output from the GA must have cost at least as good as the TSP solution. In roulette wheel selection, this is

TABLE 2. VACCINE EXAMPLE

Desc.	Walk	Cost
pre-vaccine	4, 3, 2, 4, 1, 3, 1, 2, 3	3.45
post-vaccine	4, 3, 2, 1, 4, 3, 1, 2, 3	2.58

not necessarily the case since individuals are selected at random (with probability proportionate to their respective fitness). Under roulette wheel selection one must take care to keep a record of the best individual as it may not persist across the generations. Stochastic acceptance [14] is used here for speed and ease of implementation.

Immunization. The last operator presented here is the immunization operator which is used in the immune genetic algorithm (IGA) scheme [12]. In IGA, we seek to infuse some problem-specific information at a chromosomal level. For example, in applying IGA to the TSP an example of a "vaccine" is where, for a particular chromosome, a node is chosen and its nearest neighbor is determined. If the tour encoded by this chromosome doesn't visit the nearest neighbor directly after the chosen node, then the chromosome is amended to include this edge. In many cases this operation will decrease the cost of the tour by trading some spurious edge for a better one. This works for the TSP because solutions *tend* to have edges between nearest neighbors. Thus a vaccine can be thought of as some alteration to an individual that is likely to improve its fitness. The vaccine is typically applied to individuals in the population with some probability, P_{vacc}.

For the persistent visitation problem, the walk may be comprised of many inner loops or subwalks (i.e. the walk $4,3,2,4,1,3,1,2,3$ has a subwalk $2,4,1,3,1$ which begins at site 2 and ends at 2). The basic premise of the vaccine for this problem is that subwalks ought to have a TSP structure. In larger walks, these subwalks could be nontrivial and may, themselves, contain *sub*subwalks. So, prior to rearranging the subwalk, all *sub*subwalks are removed, e.g,

$$\underbrace{4,3,\mathbf{2,4,1,3,1},2,3}_{\text{walk}} \rightarrow \underbrace{2,4,\mathbf{1,3},1}_{\text{subwalk}} \rightarrow \underbrace{2,4,1}_{\text{reduced subwalk}}$$

For very large problems there could be an arbitrary number of levels of subwalk for which rearranging to the TSP could be of benefit, however, for speed and ease of implementation, only the first level is considered. Table 2 contains an example where $m = 4$, $N = 9$, and the coordinates and weights are chosen from a uniform distribution. In this example, the subwalk $2,4,1,3,1$ is the only subwalk that gets rearranged. After the subwalk is rearranged to the TSP solution for these sites, the cost is reduced. In larger problems, there may be more than one such rearrangement and the cost reduction could be more dramatic. The downside to this vaccination approach is the need to compute many different sub-TSP solutions. In a large enough population, most of the possible combinations of sites may show up as subwalks for which the TSP must be computed. Computing every sub-TSP *a priori* is still burdensome since there are $\sum_{i=4}^{m} \binom{m}{i}$ such sub-TSPs. Even if a heuristic algorithm is used to solve these sub-TSPs (e.g. LKH [7]) the computational cost is prohibitive. The immunization operator is omitted in the computational results because the increased CPU time negates the benefit of using a GA in the first place.

GREEDY TREE SEARCH

We now turn our attention to an altogether different search technique which is included as an additional preprocessing step: the greedy tree search (GTS). Each node in the tree represents a complete walk, and its cost is the value of the weighted revisit time of the site $i^* = \arg\max_i w_i \cdot R(i)$. Without reordering a walk, the only way to reduce the cost is by inserting an additional visit to i^*. The children of a node in the tree are all of the possible walks in which an additional visit to i^* has been inserted. We do not consider identity transitions in generating the children of a node since identity transitions do not affect cost. Figure 1 shows an example with $m = 5$. The distance matrix and weights are given by,

$$D = \begin{bmatrix} 0 & .52 & .18 & .29 & .52 \\ .52 & 0 & .59 & .74 & .02 \\ .18 & .59 & 0 & .43 & .58 \\ .29 & .74 & .43 & 0 & .74 \\ .52 & .02 & .58 & .74 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} .08 & .78 & .44 & .72 & .98 \end{bmatrix}$$

The cost of the root node is dominated by site 5 (i.e $i^* = 5$), denoted by a box, thus the children each contain an additional visit to site 5. A greedy selection is made among the three children, denoted by the gray rectangle. This node's cost is also dominated by site 5 and so its children will contain an additional visit to site 5. In general, the cost is non-monotonic as the tree is traversed in greedy fashion. This is seen clearly in Figure 2 where the best cost occurs at $N = 9$ after the cost had risen. Thus we refer to the output of the greedy tree search as the walk with the best cost observed up to some maximum walk length.

Because the ordering of visits is fixed for each node, the original ordering imposed by the root node is important. A sensible choice of walk for the root node is the TSP since it is complete and optimal for $N = m$. Table 3 contains the results of 100 experiments with random coordinates and weights with $m = 6$,

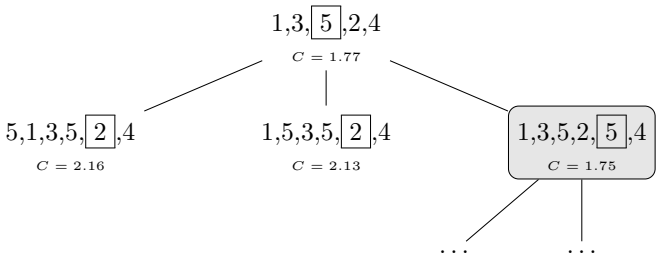


FIGURE 1. GREEDY TREE SEARCH EXAMPLE

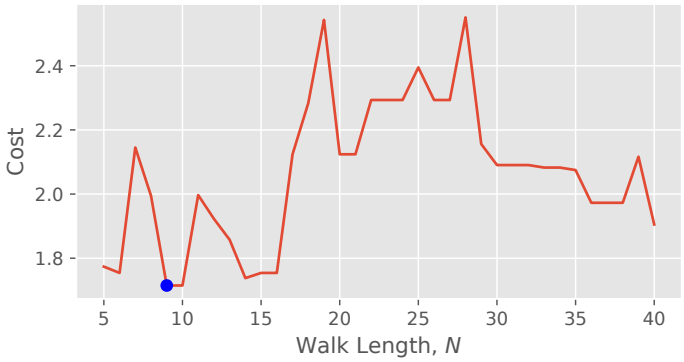


FIGURE 2. GREEDY TREE SEARCH COST HISTORY

$N_{max} = 100$. The output of the greedy tree search using the TSP as the root node is compared against 100 randomly chosen roots. Only 0.25% of the randomly initialized GTS runs resulted in a cost as good as the TSP itself, and even fewer (0.09%) beat the cost obtained by initializing GTS with the TSP. This indicates that the TSP is almost always a good root node to use in GTS.

Henceforth, any invocation of the greedy tree search uses the TSP solution as the root node. The utility of this search method is its relative speed compared to both the GA and MILP formulations, as will be seen in the sequel. For cases where the greedy tree search yields a walk whose cost is less than the cost of the TSP we essentially have a tighter upper bound which is a function of N, whereas our previous upper bound, $\bar{C}$ is static.

TABLE 3. RANDOM ROOT COSTS COMPARED TO TSP COST AND TSP AS ROOT COST

$C < \bar{C}$	$C < C_{\text{TSP Root}}$
0.25%	0.09%

RESULTS

In this section we focus on the results of two main experiments wherein the MILP methods [6], GA, and GTS are compared in terms of performance (lowest cost achieved) and CPU time. The first is a Monte Carlo experiment with $m = 6$ and $N_{max} = 10$, and the second is a more realistic scenario modeling a police patrol strategy in response to measured crime rates in San Francisco, taken from [5, 6]. We refer to the two MILP methods from [6] as *Opt*, which is the exact formulation, and *SubOpt*, which is a formulation in which the number of visits to each site is specified. The exact formulation has $O(Nm^2)$ binary variables, $O(Nm)$ real variables, and $O(Nm)$ constraints, and thus it scales very poorly. In fact, the scalability of the exact solution to persistent visitation is much worse than the TSP since the MILP not only searches over the ordering of visits but also the number of each visits for each site. Additionally, the structure of the cost function for persistent visitation is more computationally burdensome as there are many more variables to track (i.e. each site may have many subwalks that must be considered when computing $R(i)$). The specification of visits for the SubOpt scheme is determined by a heuristic algorithm described in detail in [6]. The procedure starts with $N = m$ and increments N, adding a visit to the site i^* which dominates the cost. If a site is not able to "use" the additional visit (meaning the MILP outputs a walk with an $i^* \to i^*$ leg), the next highest weighted revisit time site is given an additional visit, and so on. There is a benefit to using the SubOpt scheme compared to using the Opt scheme in terms of computational complexity. This is due to the additional constraints which force the solution to contain a certain number of visits to each site which has the effect of reducing the search space: in SubOpt, the MILP only searches over the sequence of visits. In that regard, SubOpt is much more similar to the TSP, but the cost is still quite different.

In all the experiments, the GA is run five times because it is non-deterministic. Generally, the results for the GA are aggregated in terms of the average across these five runs as well as the minimum and maximum for cost and CPU time.

Monte Carlo

The Monte Carlo experiment is comprised of 132 random cases (uniform random weights and uniform random coordinates). Table 4 contains the settings used for the genetic algorithm. In order to compare performance, we set $N = 10$ (and $N_{max} = 10$ for GTS) and examine the ratio of the costs of SubOpt, GA, and GTS with respect to Opt (see Figure 3). The bars around the GA points indicate the distribution when considering only the minimum or maximum values obtained by the five runs of the GA. For example, taking only the minimum costs for the GA, over 80% of the cases have cost within 1% of the true optimal cost given by Opt. On average, 74% of the cases are within 1% of the optimal, and taking the maximum costs (i.e. the *worst*

TABLE 4. GA SETTINGS FOR MONTE CARLO

Generations	20
Population Size	100
P_{mut}	0.01
P_{cross}	0.70
Selection	truncation
Seed	TSP

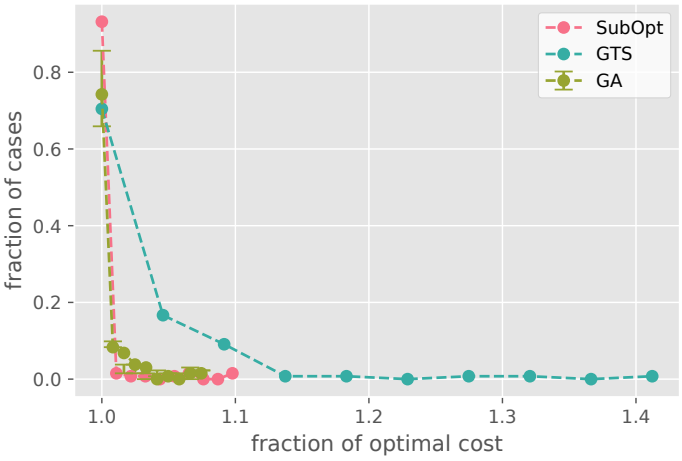


FIGURE 3. COST COMPARISON FOR MONTE CARLO

TABLE 5. COST STATISTICS FOR MONTE CARLO

	GA (Avg)	SubOpt	GTS
% Cases	74	93	70
Within (%) of Optimal	1	1	5
Worst Cost Ratio	1.08	1.10	1.46

runs), 66% of the cases are within 1% of the optimal. Note that all of the SubOpt walks are below 1.11 times the optimal cost and, on average, the GA walks are below 1.08 times the optimal cost. Interestingly, GTS finds walks within 5% of the optimal cost for 70% of the cases. On the other hand, the cost for GTS can be as bad as 1.46 times the optimal; but this cost is still bounded above by the TSP cost, $\bar{C}$. Table 5 summarizes these results.

The performance of the GA is comparable to the SubOpt method in terms of the cost of the walk returned. While the SubOpt method has more cases within 1% of the optimal cost,

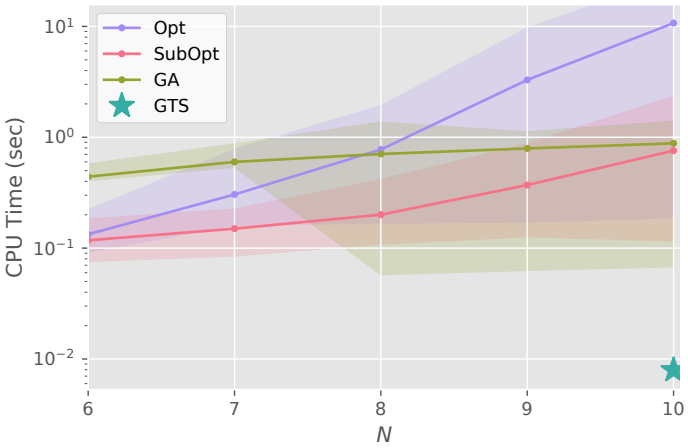


FIGURE 4. CPU TIME COMPARISON FOR MONTE CARLO

the GA has slightly better worst-case performance in this experiment.

Next we compare the CPU times for each of the methods. For the Opt, SubOpt, and GA methods, we set $N = m = 6$ and increment until $N = N_{max} = 10$ to illustrate the computational complexity with respect to N for fixed m. Figure 4 shows the CPU times as a function of N in log-scale. The shaded regions represent the variability between the minimum and maximum values for each method. Note, GTS is only run for $N = 10$ and the variability in CPU times across the Monte Carlo cases is negligible. Now it is clear that since the CPU time of GTS is over two orders of magnitude less than SubOpt and GA that it serves as a valuable preprocessing step. The output of GTS serves as a tighter upper bound than the TSP cost and may even be used as a seed walk for the initial GA population. As seen in Figure 4 both Opt and SubOpt scale poorly as N increases. Initially, the CPU time of GA is poor, but it scales much better. By $N = 10$ the CPU time of GA is very close to the CPU time of the SubOpt scheme. Presumably, if the experiment were carried out for larger N, the GA would become advantageous over either the SubOpt or Opt methods because of its scalability. The sharp drop in the minimum time for the GA is due to the fact that for at least one of the 132 Monte Carlo cases the optimal cost is equal to the lower bound, $\underline{C}$. In this case, the GA terminates immediately upon finding a walk with cost equal to $\underline{C}$ which could occur as early as generation zero.

Police Patrol

This example originally appeared in [5] and was revisited in [6] wherein the Opt and SubOpt MILP methods were applied to the problem. Twelve sites corresponding to street intersections in San Francisco are of interest due to the number of crimes that occur there (a map of the area is given in Figure 9 of [5]). Note

TABLE 6. STATIC AND VARYING GA PARAMETER SET

	Static	Varying
Generations	75	$75(1.1)^{N-12}$
Population Size	300	$300(1.1)^{N-12}$
P_{mut}	0.01	0.01
P_{cross}	0.70	0.70

TABLE 7. COMPLETE GA SETTINGS FOR ALL 5 CASES

	GA1	GA2	GA3	GA4	GA5
Params	Static	Static	Varying	Varying	Varying
Selection	Trunc.	Trunc.	Trunc.	Trunc.	Roul.
Seed	-	GTS	-	$N-1$	$N-1$

the distance matrix, from [5] is asymmetric as it represents distances traveled on a road network. The sites are weighted by the number of crimes reported to have taken place at each intersection in August 2012 [5]:

$$N_{\text{crimes}} = \{133, 90, 89, 87, 83, 83, 74, 64, 48, 43, 38, 34\}$$

If we normalize by the factor $31 \cdot 24 \cdot 3600$, the weights may be understood to represent an expected number of crimes per second. Thus by framing the problem in terms of persistent visitation, the police patrol aims to minimize the maximum number of crimes committed between visits.

Each of the four methods are compared starting for $N = 12$, incrementing N in succession. The results for Opt and SubOpt come straight from [6]. Also note, due to the exponentially rising CPU time, SubOpt was run up to $N = 21$, and Opt was run up to $N = 14$. For cases $N = 12, 13, 14$ SubOpt and Opt had the same cost. Due to the combinatorial nature of the problem, the search space for this example with $m = 12$ is vast compared to the $m = 6$ Monte Carlo cases. Thus more care is taken here to tune the parameters of the GA; in particular, the GA is run with both static and varying parameter sets. The varying parameter sets change with respect to N because the search space increases with N. Table 6 gives the parameter sets for both the static and varying case. In combination with these two cases, the choice of selection scheme and seed walks are also examined. In total, 5 cases are considered, the specification of which are presented in Table 7. All of the GA cases are seeded with the TSP in addition to what is specified. The $N-1$ seed denotes using the best walk found from the previous run as a seed. For these cases, the CPU time includes the sum of all prior runs performed leading up to N. Figures 5 and 6 contain the results for the cost and CPU time, respectively.

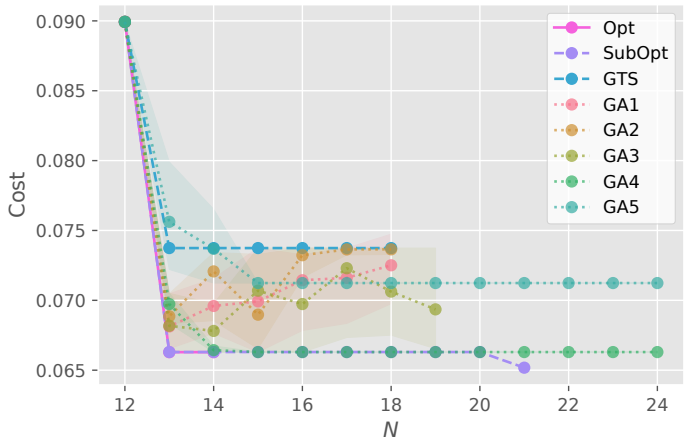


FIGURE 5. COST COMPARISON FOR POLICE PATROL EXAMPLE

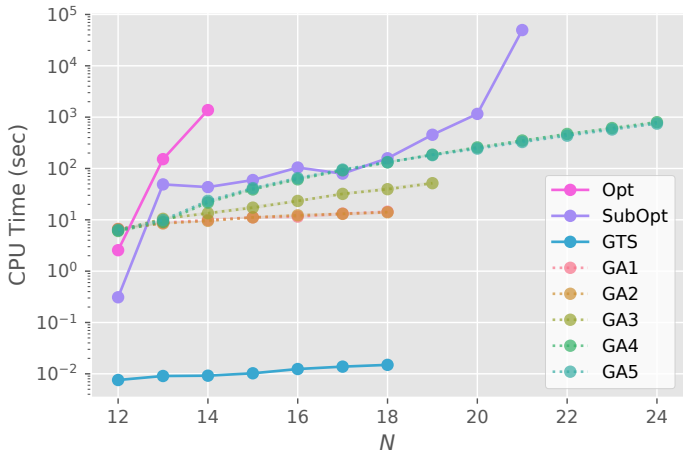


FIGURE 6. CPU COMPARISON FOR POLICE PATROL EXAMPLE

First note the shape of the cost curve for GTS in Figure 5 – the cost plateaus immediately at $N = 13$. The reason for this is that the GTS hits its minimum value at $N = 13$ and appears to stray further and further away for increasing N (see Figure 7). Interestingly, the walk obtained by GTS has a cost within 11% of the true optimal for $N = 14$ and is computed in hundredths of a second, over 5 orders of magnitude faster than Opt. Thus GTS excels at finding what may be considered a "local optimal" walk in very little time. For larger problems (e.g. involving $m \geq 30$ sites), GTS may be the only choice among these methods for finding a reasonable walk within an acceptable time. This may be

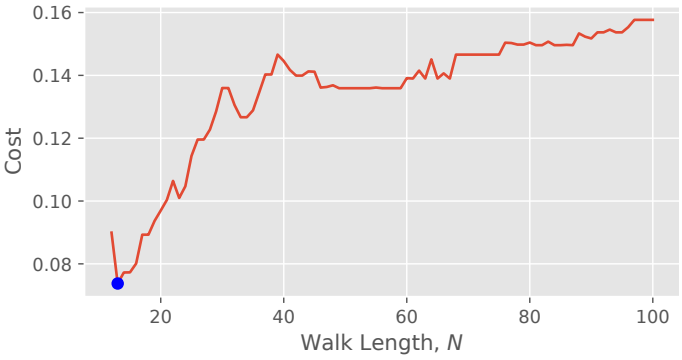
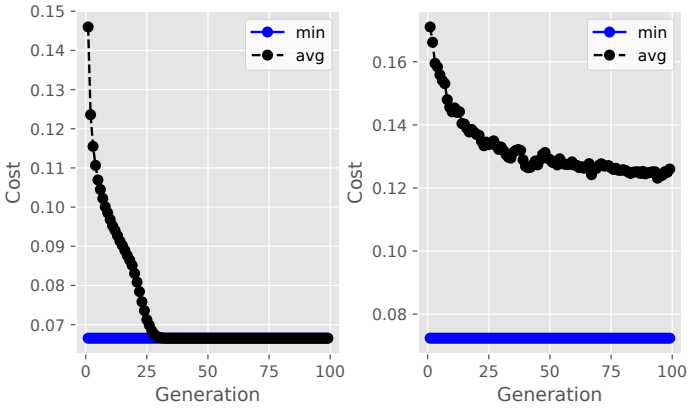


FIGURE 7. GTS COST HISTORY FOR POLICE PATROL EXAMPLE



(a) TRUNCATION (b) ROULETTE WHEEL

FIGURE 8. CONVERGENCE FOR DIFFERENT SELECTION OPERATORS

especially true for onboard applications wherein computational resources are typically limited.

The settings of GA1 most closely resemble the settings used in the Monte Carlo experiment. From Figure 5 shows that, on average, GA1 finds its best cost for $N = 13$. Because of the allowance of identity transitions in the walks, the optimal cost is monotonic non-increasing. GA1's cost trajectory, however, rises after $N = 13$ monotonically. This is likely the result of the combinatorial expansion of the search space. Incrementing N by one for this example increases the number of possible walks much more than the Monte Carlo cases. This provides some motivation for using the varying parameter set in Table 6 since the number of generations and population size are exponential in N. Both of these parameters directly affect how much of the space the GA is able to search.

For GA2, the output of GTS is used as a seed (in addition to the TSP). A similar trend to GA1 is observed in GA2's cost curve in Figure 5: the cost reaches a minimum at $N = 13$ and trends upwards after. For $N = 16, 17, 18$, the cost is very close to the GTS cost which suggests that GA2 may be converging prematurely to the GTS walk when the search space is large. Not surprisingly, the CPU time for GA1 and GA2 is essentially the same since the only difference is in how they are seeded (see Figure 6).

Usage of the varied parameter scheme increases the CPU time for GA3, GA4, and GA5 compared to GA1 and GA2. GA3 gives a slightly better performance in terms of cost than GA1 and GA2. However, as evidenced by the shaded region, GA3 tends to vary much more. At best, the GA3 matches the cost of Opt and SubOpt for $N = 13, 14$; and, at worst, GA3 is as bad as GTS for $N = 18, 19$. Thus the increased CPU time is not necessarily well spent since, for a given run, we could either get no improvement over GTS or the optimal.

Finally, both GA4 and GA5 exhibit vastly improved stability and reduced variability over the previous cases. In Figure 5 both curves quickly reach a plateau wherein the variability collapses. For example, for GA4, all five runs result in the same cost for $N = 14$. The biggest reason that GA4 and GA5 appear to settle into different costs is that the choice of selection operator has a big impact on the convergence. With truncation, the population converges rapidly, while roulette wheel selection maintains genetic diversity for much longer. Conceptually, roulette wheel selection searches the space more slowly, requiring more generations to search the space and converge. Figure 8 illustrates this point: the average cost across the population in Figure 8a collapses to the minimum just after the 25th generation for truncation whereas Figure 8b shows the diversity being maintained even at the 100th generation for roulette wheel. Thus its quite possible that GA5 would perform as well as GA4 if the number of generations and perhaps the population size were increased. The downside of fast convergence via truncation is that the true optimal is more likely to be missed in large problems.

The CPU time for GA4 and GA5 are nearly identical because their only difference is in the selection operator used. Note because stochastic acceptance is used to implement roulette wheel selection, the CPU time increase compared to truncation is negligible. Overall GA4 (varying parameters, truncation selection, and seeding with fittest individual from $N-1$th generation) exhibits the best cost performance among all of the GA configurations. Moreover, the CPU time of GA4 grows much slower than that of even the SubOpt scheme. For this particular example, GA4 offers a significant advantage over the MILP schemes especially since GA4's cost is nearly identical. In larger examples, the cost performance may become more degraded, but the CPU time savings would make this a favorable trade.

CONCLUSION

We have discussed the problem of persistent visitation for UAVs wherein the goal is to minimize the maximum weighted revisit time. There is a strong basis for applying GAs to this particular problem because of how they have been applied to the TSP, which is related, and because of the poor scaling of the MILP formulations. A novel chromosome encoding scheme was presented which, along with the genetic operators, helps to ensure feasibility throughout the population. Additionally, a simple greedy tree search was presented as a preprocessing step which can be used as an upper bound to the cost. The work in [8] suggests that the design of the chromosome and genetic operators can greatly affect the performance of a GA. There may be benefit to pursuing encoding schemes and genetic operators that embed and make use of more problem specific information (like the immunization operator given here). Another encoding scheme may, for example, encode subwalks and their sequence. The results indicate that the GA can match the cost performance of the MILP formulations for small examples with 6 sites, and nearly match for larger examples with 12 sites. Scaling of the GA in terms of CPU time is much better than the MILP methods. For large problems with 20 or more sites, the GA may be of more practical use even if the cost performance is degraded.

REFERENCES

[1] Kalyanam, K., Pachter, M., and Casbeer, D., 2017. "Average Reward Dynamic Programming Applied to a Persistent Visitation and Data Delivery Problem". In Proceedings of the ASME 2017 Dynamic Systems and Control Conference.

[2] Manyam, S. G., Rasmussen, S., Casbeer, D. W., Kalyanam, K., and Manickam, S., 2017. "Multi-UAV routing for persistent intelligence surveillance reconnaissance missions". In 2017 International Conference on Unmanned Aircraft Systems (ICUAS), pp. 573–580.

[3] Rasmussen, S., Kalyanam, K., Manyam, S., Casbeer, D., and Olsen, C., 2017. "Practical considerations for implementing an autonomous, persistent, intelligence, surveillance, and reconnaissance system". In 2017 IEEE Conference on Control Technology and Applications (CCTA), pp. 1847–1854.

[4] Pasqualetti, F., Durham, J. W., and Bullo, F., 2012. "Cooperative Patrolling via Weighted Tours: Performance Analysis and Distributed Algorithms". *IEEE Transactions on Robotics,* ***28***(5), Oct., pp. 1181–1188.

[5] Alamdari, S., Fata, E., and Smith, S. L., 2014. "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations". *The International Journal of Robotics Research,* ***33***(1), Jan., pp. 138–154.

[6] Kalyanam, K., Manyam, S., Von Moll, A., Casbeer, D., and Pachter, M., 2018. "Scalable and Exact MILP Methods for UAV Persistent Visitation Problem". In Conference on Control Technology and Applications, Vol. Manuscript submitted for publication, IEEE. `https://sites.google.com/site/krishnakalyanam/ccta2018.pdf`.

[7] Helsgaun, K., 2000. "An effective implementation of the Lin–Kernighan traveling salesman heuristic". *European Journal of Operational Research,* ***126***(1), Oct., pp. 106–130.

[8] Potvin, J.-Y., 1996. "Genetic algorithms for the traveling salesman problem". *Annals of Operations Research,* ***63***(3), June, pp. 337–370.

[9] Ray, S. S., Bandyopadhyay, S., and Pal, S. K., 2004. "New operators of genetic algorithms for traveling salesman problem". In Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004., Vol. 2, pp. 497–500 Vol.2.

[10] Takahashi, R., 2005. "Solving the traveling salesman problem through genetic algorithms with changing crossover operators". In Fourth International Conference on Machine Learning and Applications (ICMLA'05), pp. 6 pp.–.

[11] Frahadnia, F., 2009. "A New Method Based on Genetic Algorithms for Solving Traveling Salesman Problem". In Modelling and Simulation 2009 International Conference on Computational Intelligence, pp. 11–16.

[12] Jiao, L., and Wang, L., 2000. "A novel genetic algorithm based on immunity". *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans,* ***30***(5), Sept., pp. 552–561.

[13] Grefenstette, J., Gopal, R., J. Rosmaita, B., and Van Gucht, D., 1985. "Genetic Algorithms for the Traveling Salesman Problem". In Proceedings of the First Internationa Conference on Genetic Algoroithms and Their Applications.

[14] Lipowski, A., and Lipowska, D., 2012. "Roulette-wheel selection via stochastic acceptance". *Physica A: Statistical Mechanics and its Applications,* ***391***(6), Mar., pp. 2193–2196.

View publication stats